Bolt: Faster SNARKs from Sketched Codes

Kobi Gurkan* Andrija Novakovic† Ron D. Rothblum‡

May 20, 2026

Abstract

We introduce *Bolt*, a new Multilinear Polynomial Commitment Scheme (MLPCS) designed for high-performance SNARKs. Bolt is geared towards SNARKs for large computations, in which prover speed is paramount but one can afford slightly larger proofs. The construction is based on the code-switching paradigm; our core technical contribution is a new "proof-system friendly" error-correcting code with extremely efficient encoding both asymptotically and concretely.

A theory-oriented instantiation of Bolt achieves a commitment time of approximately $(3 + \varepsilon) \cdot N$ field additions plus a Merkle Tree hash computation of size $(1 + \varepsilon) \cdot N$ field elements, where N is the size of the multilinear polynomial and $\varepsilon > 0$ is arbitrarily small. The prior state-of-the-art, Blaze (Brehm et al., Eurocrypt 2025) used $8N$ additions and a $4N$ size Merkle hash.

Concretely efficient instantiations of Bolt demonstrate that these asymptotic gains translate into substantial real-world speedups. Our benchmarks show that for $N = 2^{30}$ over the field $\mathrm{GF}(2^{32})$ Bolt's commitment time is roughly 3×–4× faster than Reed-Solomon based schemes, depending on the specific parameterization. In the slower variant, the proof-size is ∼2MB. Bolt also offers better commitment time *and* especially proof size compared to recent linear-time schemes. For example, its commitment time is about 20% faster than Brakedown (Golovnev et al., Crypto 2023) with a 5×–15× smaller proof.

*Bain Capital Crypto. kgurkan@baincapital.com
†Bain Capital Crypto. anovakovic@baincapital.com
‡Succinct. Email: ron@succinct.xyz

Contents

1 Introduction **3**
- 1.1 Our Contribution . . . 3
- 1.2 Technical Overview . . . 6
- 1.3 Open Questions . . . 11
- 1.4 Organization . . . 12

2 Preliminaries **12**
- 2.1 Multilinear Extension . . . 12
- 2.2 Interactive Proofs and IOPs . . . 13
- 2.3 Coding Theory . . . 14

3 Sketched Coding **15**

4 IOP for Sketched Codes **16**
- 4.1 Proof of Lemma 4.1 . . . 17

5 IOP for Interleaved Sketched Codes **19**
- 5.1 Proof of Lemma 5.1 . . . 20

6 Random LDPCs **21**
- 6.1 Random LDPCs over Bits . . . 21
- 6.2 Random LDPC over Large Fields . . . 23

7 Instantiations **25**
- 7.1 A Fast Code: Proof of Theorem 1.2 . . . 25
- 7.2 An Asymptotically Efficient PCS: Proof Sketch of Theorem 1.1 . . . 25
- 7.3 The Concrete Instantiation . . . 26

8 Benchmarks and Evaluation **26**

1 Introduction

Succinct Non-Interactive Arguments of Knowledge (SNARKs) have evolved from deep within the theory of cryptography and complexity-theory into critical infrastructure for verifiable computation and blockchain scalability. As these systems are deployed to prove increasingly complex statements, the computational cost of the prover has emerged as a primary bottleneck.[1]

Polynomial commitment schemes (PCS) are a dominant cost driver for efficient SNARKs. A PCS allows the prover to provide a succinct commitment to a large polynomial P (which, in a SNARK, typically encodes a computation trace) so that later it can prove statements of the form "$P(x) = y$". We focus on *multilinear commitments* in which P is a multilinear polynomial (i.e., a multivariate polynomial with degree at most 1 in each variable).

Existing multilinear polynomial commitments schemes (MLPCSs) generally fall into two categories: those based purely on hash functions (usually analyzed in the random oracle model), and those based on more structured cryptography such as hardness of discrete log or of lattice problems. In this work we restrict our attention to hash-based schemes. Within this family there are two main approaches: those based on Reed-Solomon (RS) codes (e.g., [AHIV23, BBHR18, ZCF24, ACFY24, ACFY25, NA25]) and those based on linear-time encodable codes [BCG+17, BCG20, GLS+23, RR25, BCF+25, NST25, BMMS25a, ARR25, BMMS25b, BFRW25]. RS-based schemes offer more compact proofs but suffer from the concrete overhead of the Fast Fourier Transform (FFTs), which becomes the bottleneck for large instances. Conversely, the newest linear-time schemes [BMMS25b, BFRW25] offer essentially optimal asymptotics but tend to have concretely larger proofs.

In this work we ask: to what extreme can we take the linear-time encoding approach? As linear-time proving becomes commonplace, we are no longer content with asymptotic bounds alone. Rather, we aim to minimize the *computational rate* of provers, that is, the constant c required for a PCS or SNARK to execute in time $c \cdot N$, for inputs of size N. To do so we are willing to have a moderately larger, but still reasonable, proof-size.

1.1 Our Contribution

We construct a new Multilinear Polynomial Commitment Scheme (MLPCS), called *Bolt*, designed specifically for high-performance SNARKs. Bolt is geared towards SNARKs for large computations, in which prover speed is paramount but one can afford slightly larger proofs. We believe that Bolt is particularly attractive for client-side proving in which the FFTs tend to be a major bottleneck, and in which trading off proof-size for better prover latency is usually desirable, let alone if the proof-size can be further reduced via recursion with another proof-system optimized for small proofs.

Bolt follows the *code-switching* paradigm [RR24], specifically in a similar way to how it was used in [BCF+25]. Our core technical contribution is the construction of a new error-correcting code — the *Sketched Code* — that is extremely efficient both asymptotically and concretely. Beyond the extremely fast encoding, we show that the sketched code has several important proof-system friendly features on which we elaborate in Section 1.1.1 below.

The Asymptotic Angle. Asymptotically, over binary fields, Bolt achieves a commitment time of $(3+\epsilon) \cdot N$ field additions plus an additional Merkle tree computation on $(1+\epsilon) \cdot N$ field elements,

[1] See, e.g., `https://zkevm.ethereum.foundation`.

where N is the size of the multilinear polynomial and $\epsilon > 0$ is arbitrarily small. This improves significantly over the prior state-of-the-art, Blaze [BCF+25], which required $8N$ additions.[2]

We summarize our main asymptotic results in the following theorem:

Theorem 1.1 (Informally Stated)**.** *Let $\mathbb{F}$ be a binary field. For every $\epsilon > 0$, there exists a multilinear polynomial commitment scheme for $\mathbb{F}$, in the random oracle model, with the following parameters:*

- ***Soundness Error:*** $2^{-\lambda}$.
- ***Commitment Time:*** $(3+\varepsilon) \cdot N$ *field additions and* $(1+\varepsilon) \cdot N$ *Merkle hash computation.*
- ***Evaluation Proof:*** $(2 + o(1)) \cdot N$ *field operations.*
- ***Proof Size:*** $O(\lambda \cdot \log(N) \cdot \log\log(N))$.
- ***Verifier Time:*** $O(\lambda \cdot \log(N) \cdot \log\log(N))$.

The most notable aspect of Theorem 1.1 is the roughly $3N$ commitment-time, improving over the $8N$ achieved in [BCF+25]. The other parameters are similar to those achieved by state-of-the-art IOPs [ACFY25, BMMS25a, BMMS25b, BFRW25] (actually, the proof-size and verification time are off, by an $O(\log\log N)$ factor from [BMMS25b, BFRW25], but could be improved by utilizing similar techniques, which we avoid for sake of simplicity).

Concrete Performance. The big-O notation in the proof-size in Theorem 1.1 hides large constants and so in the concrete instantiation we choose different parameters, to allow for reasonably sized proofs.

Our implementation demonstrates that the asymptotic gains translate into substantial real-world speedups. In our implementation we consider two main settings: the first optimizes for prover speed, but has larger proofs, and the second has a slightly less efficient proof but better proof size.

Specifically, for $N = 2^{30}$ over a 32-bit binary field, Bolt achieves a commitment time between $3\times$–$4\times$ faster than an analogous Reed-Solomon based commitment [NA25]. The proof-size ranges between about 2MB in the leaner version and 13MB in the fast version. These sizes are larger than in Reed-Solomon based schemes and so we only recommend using Bolt in contexts where proof-size is not a major issue, or if it is plausible to use recursion to compress the proof.

Bolt is faster than previous linear-time schemes, while offering substantially smaller proofs, and other additional advantages. Bolt's commitment is about 20% faster than Brakedown's [GLS+23] commitment and $20\% - 45\%$ faster than that of Blaze, with a proof that is about $5\times$–$15\times$ smaller than Brakedown's.[3]

We refer the reader to Section 8 (and specifically Table 1) for the specifics of our benchmark methodology, results, and additional details.

[2] Blaze also uses a Merkle hash computation of size $4N$, but this can be alleviated via code puncturing (see [BCF+25, Section 5.4]). In contrast, the amount of additions seems inherent.

[3] Bolt has amortized and non-amortized variants. The non-amortized variant utilizes the Mulperm protocol [BCD25] for which an implementation is not yet available. We carefully estimated the cost of that protocol and the evaluation proof time quoted above is based on that estimate, see Remark 8.1 for details.

1.1.1 Sketched Codes

As alluded to above, our core technical contribution is the design of a new efficiently encodable error correcting code. The simple approach, inspired by classical results in coding theory [GDP73, Spi96, FLY22], allows one to compose error-correcting codes that have different properties together to obtain a new code with appealing properties — first and foremost, extremely fast encoding.

From an asymptotic perspective, the code, which is defined over the binary alphabet, can be encoded by a circuit of size $(3+\varepsilon)\cdot k$ for messages of size k. It has a small but positive constant relative distance $\delta > 0$. More accurately, similarly to the codes that underlie [GLS+23, BCF+25] the code is actually a collection of codes. The guarantee is that a random code from the family has distance δ with high probability. We emphasize that the code just needs to be sampled once and can then be used for eternity. The code is guaranteed to have "good" minimal distance, unless some extremely unlikely event happened during the setup phase.

Theorem 1.2. *For any desired rate parameter $r > 0$ and $\varepsilon > 0$, there exists $\delta = \delta(r, \varepsilon) > 0$ and a family of error-correcting codes $\{C : \{0,1\}^k \to \{0,1\}^n\}$ of rate r, that are computable by circuits of size $(3+\varepsilon)\cdot k$, such that a random code from the family has distance δ with all but $O(1/n)$ probability.*

Similarly to Blaze [BCF+25, Section 1.2.2] the error probability can be reduced to any inverse polynomial error by invoking a more complicated sampling procedure, which runs in time inversely proportional to the desired error. See Remark 6.2 for details.

Beyond the efficient encoding, the sketched code that we construct has key properties, that are extremely useful in the construction of efficient proofs:

1. We construct an efficient *proximity test* for membership in the sketched code. As we show later (in Section 1.2), this proximity test drastically improves the efficiency of the evaluation proof of our PCS.

2. The code natively has a high rate, supporting rates arbitrarily close to 1. Such high rate codes are useful for proof-systems as they can (1) drastically reduce the amount of hashing required (recall that in hash based proofs the prover typically needs to generate a Merkle hash of the codewords), and (2) reduce the memory footprint of encoding. While the Reed-Solomon code in principle supports rates arbitrarily close to 1, for rates above 1/2 it becomes unfriendly to algorithms that rely on the underlying group structure (e.g., FFT and proximity tests such as FRI and WHIR). Also note that the RAA ensemble considered in [BCF+25] has a rate of 1/4.

   It is worth pointing out that any code can be made to have a high rate via *code puncturing*, but this approach seems quite wasteful and may be incompatible with the accompanying proof-system.

3. Lastly, the sketched code is natively *systematic*. This means that the encoding of a message m always starts with the message m itself.

   Having a systematic encoding is extremely compelling in a SNARK application. The prover in a SNARK typically needs to encode the computation trace and then perform a Merkle hash of the codeword. When using a systematic code, since Merkle hashing can be done in a streaming manner, the prover can start hashing in parallel to processing the computation trace.

While it is possible to transform any code (in particular the RAA or Reed-Solomon code) to be systematic, this usually introduces significant overhead.

Concurrent Work. A concurrent and independent work by Qu et al. [QGZ26] introduces *Lightning*, a polynomial commitment scheme which similarly proposes to encode a sparse linear sketch of the message.[4] Both systems use sketched codes to construct more efficient polynomial commitment schemes but the details differ.

One of the primary differences is in the evaluation proof. Lightning uses a Ligero/Brakedown approach and uses the sketched codes as a drop-in replacement for the Spielman-type code. Consequentally Lightning has a roughly $O(\sqrt{n})$ proof-size. Bolt uses a tailored code-switching approach to obtain a poly-logarithmic proof-size. The initial version of Bolt focused primarily on binary fields whereas Lightning is field agnostic. The current updated version of Bolt also considers the case of general fields.

Concretely, both schemes obtain a faster commitment than Brakedown, but whereas Lightning has a larger proof than Brakedown, Bolt's proof is about $5\times$ to $15\times$ smaller. Furthermore, our work formally studies the precise asymptotic computational rate of this paradigm, demonstrating that a theory-oriented parameterization of Bolt achieves an encoding dominated by $3N$ XORs.

1.2 Technical Overview

Our goal is to construct a multilinear polynomial commitment scheme (MLPCS) over a finite field $\mathbb{F}$. We do so by constructing a similar scheme in the interactive oracle proof model, aka an interactive oracle PCS (IOPCS) [DP24]. The IOPCS can be compiled into a cryptographic MLPCS and/or a SNARK by composing it with a vector commitment (e.g., via Merkle hashing, see [CY24] for details). An IOPCS consists of two protocols: a commitment protocol and an evaluation protocol. In the commitment protocol we are given as input a description of a vector $x \in \mathbb{F}^k$, which we associate with a function $x : \{0,1\}^{\log k} \to \mathbb{F}$ in the natural way, and the output is a "commitment oracle".[5] In the evaluation phase, the prover and verifier are given as input $z \in \mathbb{F}^{\log k}$ and $v \in \mathbb{F}$ and the goal is to convince the verifier that $\hat{x}(z) = v$, where throughout this work $\hat{x}$ denotes the multilinear extension of x (i.e., the unique multilinear polynomial that agrees with x on $\{0,1\}^{\log k}$, see Section 2.1). The evaluation phase may consist of both oracle messages and non-oracle (i.e., direct and explicit) messages.

Our construction of an IOPCS is based on the code-switching technique [RR24] as used by Brehm *et al.* [BCF+25]. We start by recalling their approach.

1.2.1 Review: Multilinear Commitments via Random Linear Combinations

Recall that an error correcting code $C : \mathbb{F}^k \to \mathbb{F}^n$ is an injective map with the property that any two codewords $C(x), C(x')$, for distinct messages $x, x' \in \mathbb{F}^k$, must be at least δ-far from each other. The parameter δ is called the distance of the code and rate of the code is k/n. We focus exclusively on linear codes which simply means that C is a linear function over $\mathbb{F}$.

Suppose we are given a code $C : \mathbb{F}^k \to \mathbb{F}^n$ that has an extremely efficient encoding procedure. Brehm *et al.* (following [AHIV23, GLS+23]) suggest to view the message we wish to commit to as a

[4] To the best of our knowledge the names of the two schemes is a coincidence.

[5] One may allow also the commitment phase to include interaction as proposed in [BFRW25]. Such interactive commitments seem necessary when working outside the unique decoding regime, which we avoid doing in this work.

$k \times t$ size matrix $X \in \mathbb{F}^{k \times t}$, and then use C to encode each of its columns.[6] The resulting codeword can be viewed as a length n codeword over the alphabet $\mathbb{F}^t$, and is referred to as an *interleaved codeword*. The commitment to X is the interleaved encoding $C^t(X)$.

Suppose that the prover sends as the codeword the matrix A, which allegedly should be equal to $C^t(X)$, for some message $X \in \mathbb{F}^{k \times t}$. In the evaluation proof, recall that we are given as input a point $z \in \mathbb{F}^{\log(k)+\log(t)}$ and value $v \in \mathbb{F}$ and the goal is to convince the verifier that $\hat{X}(z) = v$ (where we view X as a function $X : \{0,1\}^{\log k} \times \{0,1\}^{\log t} \to \mathbb{F}$ in the natural way). The prover first decomposes the claim about X into a claim about its constituent columns. That is, it views z as $z = (z_1, z_2) \in \mathbb{F}^{\log k} \times \mathbb{F}^{\log t}$ and sends (explicitly) to the verifier the function $g : \{0,1\}^{\log t} \to \mathbb{F}$ defined as $g(*) = \hat{X}(z_1, *)$. The verifier now checks that $\hat{g}(z_2) = v$, which should be true in an honest execution since $\hat{g}(z_2) = \hat{X}(z_1, z_2) = v$.

Thus, if the original claim about X is false, the prover must lie and send an incorrect $\tilde{g} \neq g$. In order to detect this, the verifier now chooses a random linear combination $r \in \mathbb{F}^t$ of the columns. Consider the vector Ar. In case A was honestly generated then $A = C^t(X)$ and we have that $Ar = C^t(X)r = C(Xr)$ and $\widehat{Xr}(z_1) = \hat{X}(z_1, r) = \hat{g}(r)$. Thus, the verifier would like to check that Ar is a valid encoding of a message x and, moreover, that $\hat{x}(z_1) = \hat{g}(r)$.

Via the proximity gap theorems for general linear codes [RVW13, AHIV23, BKS18] we can actually guarantee that for any matrix A to which the prover initially committed the following gap phenomenon occurs:

- A is close to an interleaved codeword $C^t(X)$; or
- The random linear combination Ar is far from being a valid codeword (with high probability over r).

Thus, intuitively, it suffices for the verifier to run a proof of proximity on Ar and check that it is close to some valid codeword $C(x)$ and, moreover, that $\hat{x}(z_1) = \hat{g}(r)$. The proximity test on $a = Ar$ is done by emulating each query to a by reading the entire corresponding row of A.

The crucial point is that the resulting random linear combination x is smaller, by a factor of t, than the original message, and so we can afford to run a fairly expensive proximity test on it. Hence, to obtain an efficient multilinear PCS we need two components:

1. A code C with an extremely efficient encoding procedure.
2. An efficient procedure for checking proximity to a single copy of C (with an additional multilinear evaluation constraint).

1.2.2 Sketched Codes

The core of our efficiency comes from our new error-correcting code, which we call the *sketched code*. We proceed to describe the simple construction, which is inspired by classical techniques from coding theory [GDP73, Spi96]. Let $\alpha, \gamma \in (0,1)$ be parameters. We utilize two main components:

1. A matrix $H \in \{0,1\}^{\alpha k \times k}$ whose kernel does not contain low-weight vectors. That is, for any $x \in \{0,1\}^k$ of Hamming weight at most γk, it holds that $Hx \neq 0$. Put differently, H is the parity check matrix of a code of distance at least γ.

[6] Many of the prior works, such as [AHIV23], view the message as a $t \times k$ matrix and encode the rows. We follow the "column-encoding" route (which is popular in the applied community) as it is more convenient with most of the linear algebra to come.

Specifically, we use the fact, due to Gallager [Gal62, Gal63], that a *random* sparse matrix is a good parity check matrix with high probability (the associated codes are called random low-density parity check codes, or random LDPCs for short).

2. Another linear error-correcting code $C : \{0,1\}^{\alpha k} \to \{0,1\}^{(\alpha/\rho)k}$ for messages of length αk with rate $\rho > 0$ and relative distance $\delta > 0$.

The sketched code C_H is defined by encoding a message $x \in \{0,1\}^k$ as $C_H(x) = (x, C(Hx))$. The intuition for why C_H has large distance is simple. First observe that C is a linear code and so it suffices to show that it does not have low weight codewords. Fix a message $x \in \mathbb{F}^k$. There are two options:

1. If x has high weight: then so does $C_H(x)$ since we include x in the first component of the codeword.

2. If x has low weight then by the parity check property, we know that $Hx \neq 0$. Therefore, by the distance of C we have that $C(Hx)$, the second component in the codeword, has high weight.

The above yields a code defined over bits. To obtain a code over a larger binary field, one can utilize bit slicing — that is, work over a binary extension field and encode each field element by separately encoding the individual bits.

Thus, the parity check matrix allows us to compress the given message into a *sketch*, which is then encoded under another code. This is beneficial since we can now encode the shorter sketch cheaply even using a slower code. As an additional benefit, even if the code C is non-systematic, we automatically get that the sketched code is systematic which, as noted above, can be very helpful in obtaining a fast SNARK prover.[7]

We consider both an asymptotic and concrete instantiation of the above approach.

The Asymptotic Regime. In this regime, we take the base code C to be a linear-time encodable code with encoding time ck' for messages of size k', for some constant $c \geq 1$. In this case we set the parameter $\alpha = \varepsilon/c$, for a small $\varepsilon > 0$, so that the encoding time of C for messages of size αk is merely εk. If we take H to be a super-sparse random matrix with three 1's in each column, then computing Hx can be done in time $3k$. Remarkably, Gallager's result shows that even such a sparse matrix is a parity check matrix of a code with some constant $\delta > 0$ distance (with inverse polynomial probability over the choice of H).[8]

By encoding the sketch using the code C we we get that the encoding time is $3k + c \cdot \alpha k = (3+\varepsilon)k$ as desired.

[7] Note that often codes can be made systematic but there is an associated cost. For example, to make the Reed-Solomon code systematic one needs to do an inverse FFT before performing the FFT (which increases the encoding cost by 50% in the popular rate 1/2 regime). RAA codes used in [BCF+25] are non-systematic. As Baweja *et al.* [BMMS25a] point out, the code can be made systematic by prepending the message to the codeword, but this approach is purely theoretical since doing so significantly deteriorates both the rate and distance of the code.

[8] Unfortunately, Gallager's result is tight in the sense that reducing the sparsity further leads to poor distance, see [Gal63, Theorem 2.5].

The Concrete Instantiation. Here we make a few choices that differ from the asymptotic ones. First, we do not take H to be maximally sparse as in the asymptotic case, so as not to make the distance δ be too small (recall that small distance translates to a large constant overhead in the proof size). For our first instantiation that targets maximum performance, **bolt-max**, we set our shrinkage factor to be $\alpha = 1/8$ and the number of ones per row of the matrix to also be 8. Building on Gallager's analysis we show that the resulting parity check matrix induces a code of distance at least 0.013 (with high probability).[9] For our second instantiation that targets minimum proof size, **bolt-min**, we similarly set our shrinkage factor to be $\alpha = 1/8$ and the number of non-zero elements per row of the matrix to be 16, where elements are drawn from $\mathrm{GF}(2^{32})$. The analysis in [YHC+11] shows the resulting code has a distance of at least 0.096 (with high probability).

As for the code C, we simply take the Reed-Solomon code, with rate $\rho = 1/2$. We observe that due to the 8× shrinkage factor, we will be encoding relatively small messages. For such messages the Reed-Solomon encoding is already extremely fast and so there is no point to employ more elaborate techniques. Immediately switching to the Reed-Solomon code will also help us substantially when designing a proximity test for the sketched code.

For instance, consider the PCS application for a 30-variate multilinear over $\mathrm{GF}(2^{32})$, corresponding to committing to $N = 2^{30}$ field elements). In this case we take an interleaving factor of $t = 2^7$. This means that we need the code C_H to encode t messages of size $k = N/t = 2^{23}$ each. With the shrinkage factor 8×, this means that the encoding involves $t = 2^7$ FFTs of size $\alpha k \cdot \rho^{-1} = 2^{21}$ each. Compared to just an interleaved Reed-Solomon encoding (as in [AHIV23, NA25], this reduces the total amount of FFT in the encoding by 8×, at the cost of having to compute H, which requires $8N$ XOR operations (but is actually more dominated by I/O).

On the Piece-wise Distance of the Sketched Code. As noted above, the sketched codes have the following distance guarantee. Every codeword $(x, y) \in \mathbb{F}^k \times \mathbb{F}^{(\alpha/\rho)k}$ has the property that either x has relative weight at least γ or y has relative weight at least δ, where δ is the distance of the underlying code C. Hence, the codeword is guaranteed to have weight at least

$$\frac{\min(\gamma, \delta \cdot (\alpha/\rho))}{1 + (\alpha/\rho)}.$$

For our concrete instantiation, our fast variant (aka bolt-max) has distance 0.01 and the one with the shorter proof (aka bolt-min) has distance roughly 0.07.

However, we observe that we can actually work directly with the "piecewise" distance guaranteed above. Doing so allows us to work with the distances $\gamma = 0.096$ and $\gamma = 0.013$, respectively, of the first half and reduces the number of queries that we have to do to that part by about 25%.

1.2.3 Proving Proximity to Sketched Codes

Thus, in our PCS we encode the data using an interleaved sketched code. But what about the evaluation proof? Recall that the goal in an evaluation proof is to convince the verifier that the polynomial that we committed to satisfies a multilinear polynomial evaluation constraint.

Using the random linear combination apporach discussed in Section 1.2.1, the question then boils down to a proof of proximity for a single instance of the sketched code (more precisely we also

[9] Observe that this is fairly close to the GV bound, which for the current setting of parameters, is roughly 0.0174. In our concrete instantiation, since we use 8 ones ones per row, the probability that the code fails to have distance 0.013 is bounded by $O(1/n^6)$. See Section 6 for details.

need to support a multilinear evaluation claim on the underlying message, but we defer the details on that to Remark 1.3 below).

Thus, we want to check that a given oracle $(x, y) \in \mathbb{F}^k \times^{(\alpha/\rho)k}$ is close to a codeword $C_H(x')$, for some $x' \in \mathbb{F}^k$.

Recall that in our construction y should be a valid codeword under the base code C. Thus, as our first step, we run a proximity test of y. In our concrete instantiation we use the Reed-Solomon code as our base code and so we can run protocols such as WHIR [ACFY25] or Ligerito [NA25], which are specifically designed for this task.

Thus, we can ensure that y is close to some encoding $C(u)$ for a message $u \in \mathbb{F}^{\alpha k}$. Our next goal is to check that x is close to $H^{-1}(u)$. A natural approach is to just have the verifier choose a few of the sparse constraints induced by u via H and check that they are satisfied by x. Intuitively, it seems to makes sense that if x satisfies many random constraints then it is close to a vector that satisfies all of them. Unfortunately, as shown by [BHR05] this intuition is actually false. Thus, we need to find a different way of checking consistency between x and u.

To do so, we follow the code-switching idea from [RR24], as implemented in [BCF+25]. Specifically, the prover sends a new *fresh* encoding $w = \mathsf{RS}(x)$ of x under the Reed-Solomon code (with rate say $1/2$). Thus, it suffices to check that:

1. w is close to a Reed-Solomon codeword $\mathsf{RS}(m)$.
2. x is close to m.
3. $Hm = u$.

Before describing how to perform these checks, it is worthwhile to pause and wonder what is the point of re-encoding x with a Reed-Solomon code. Wasn't the entire purpose to avoid doing so? The point, is that we are only doing this re-encoding *after* the random linear combination was applied to the committed interleaved codeword. Hence, the prover can encode the t columns using the highly efficient code C_H, and now only needs to encode the *single* random linear combination using the Reed-Solomon code.

Returning to our checks, it turns out that all three can be solved using existing techniques. The first can be directly checked using Reed-Solomon proximity checks such as WHIR or Ligerito. The second can be done by first doing spot checks — i.e., reading roughly λ/γ random coordinates of x and checking that m satisfies them, where λ is the security parameter. The checks on m can be first batched using sumcheck based techniques [RR24, CBBZ23] and then incorporated into protocols such as WHIR/Ligerito.

As for the third check, note that the verifier has access to w which is close to $\mathsf{RS}(x')$ and y which is close to $\mathsf{RS}(u)$ and needs to check that $Hm = u$. This is exactly the problem solved by *lookup protocols*. Specifically, we use the recent efficient protocol due to Bünz *et al.* [BCD25].

Remark 1.3 (Supporting a Multilinear Evaluation Proof)**.** *In the above description we focused only on the proximity check to C_H and ignored the additional multilinear evaluation that we need to support. That is, not only do we need to check that (x, y) is close to a valid codeword $C_H(x')$, but moreover, that the multilinear extension of x' at a given point z is equal to a value v.*

Recall that in the protocol the prover sends a fresh encoding $\mathsf{RS}(x)$. To handle the multilinear evaluation claim we simply incorporate this claim when batching the spot checks via the protocol of [RR24, CBBZ23].

Remark 1.4. *Throughout this work we focus exclusively on the unique decoding regime. In all of the Reed-Solomon proximity checks described above, we could in principle utilize proximity gaps in the list decoding regime [BCI+23, BCH+25]. This would require adding an "out-of-domain" sampling step and would only slightly reduce our query complexity, since the query complexity will be dominated by the spot checks to x.*

A far more noticeable improvement would be obtained if good list-decoding proximity gap bounds could be shown for random LDPCs, and we leave this as a fascinating direction for future improvement, see Section 1.3.

Cheaper Amortized Evaluation Proof. The most expensive part of the evaluation proof turns out to be the lookup argument of [BCD25]. We note however that the cost can be amortized as follows.

Recall that we use their protocol as follows: the verifier is given oracle access to strings that are respectively close to $\mathsf{RS}(m)$ and $\mathsf{RS}(u)$ and its goal is to check that $Hm = u$. Using the sumcheck protocol, this can be reduced to multilinear evaluation claims respectively to m, u and H. The claims about m (resp., u) can be incorporated directly into the proximity checks that we run on w (resp., y). Thus, we are left with a multilinear evaluation claim about the sparse matrix H.

We make two observations about this claim:

- First, it is totally independent of the committed message. Computing it, or proving correctness of its evaluation, could potentially be outsourced to a much more powerful server than what the committing player has.
- Second, if we run several evaluation proofs, even accross different committed polynomials, we end up with multiple multilinear evaluation claims about the same matrix H. Using the sumcheck protocol we can batch these multiple claims to a single multilinear evaluation claim about H, and then only run the [BCD25] protocol once on the batched instance.

1.3 Open Questions

Our work leaves several fascinating open questions:

1. On the theoretical side, what is the *optimal* computation rate of encoding? and of proving? in particular can one construct either one with less than $3\times$ overhead?

   More generally, what is the best tradeoff between computation rate and distance in the case of codes, or with queries in the case of IOPs?

2. Can analogs of Theorems 1.1 and 1.2 that use a *deterministic* code be constructed? Perhaps via expanders as in [Spi96]?

3. As mentioned above, our works gives us a concrete motivation for studying proximity gaps for random LDPC codes. This is somewhat surprising because LDPC codes do not offer efficient encoding. However, the sketched code construction shows that they are nevertheless extremely useful. A promising direction is [GG25] (building on [MRR+20]) which gives, among other results, proximity gaps for random LDPCs *up to capacity*, but requires a larger sparsity than what we can afford.

4. A compelling approach to constructing arguments with extremely fast provers *and* small proofs is to compose a fast prover protocol such as ours, with one that has a fast verifier. In order to do so in practice, we need to be able to prove Bolt's verification circuit using a verifier-efficient proof-system, without too much prover overhead.

   We leave open the question of realizing a concretely efficient proof recursion using Bolt.

5. On a related note, Bolt does not provide hiding, aka *zero-knowledge*, guarantees. Indeed, zero-knowledge seems to be at odds with having a systematic encoding. One potential way to get around this is by recursing using a zero-knowledge proof.

1.4 Organization

We start with preliminaries in Section 2. We present the Sketched Code construction in Section 3. Then, in Section 4 we give the IOP for testing proximity to a single sketched codeword and in Section 5 to an interleaved codeword. In Section 6 we present Gallager's random LDPC codes. In Section 7 we present our theoretical and concrete instantiations. Finally, benchmarks are reported in Section 8

2 Preliminaries

Let $\mathbb{F}$ be a finite field. We denote the Hamming weight (i.e., number of non-zero entries) of a vector $x \in \mathbb{F}^n$ by $\mathrm{hw}(x)$ and say that x has weight w. For two equal length vectors $x, x' \in \mathbb{F}^n$ we denote their relative Hamming distance by $\Delta(x, x') = \mathrm{hw}(x - x')/n$.

2.1 Multilinear Extension

Let $\mathbb{F}$ be a finite field and k be a positive integer. Any function $f : \{0,1\}^k \to \mathbb{F}$ defined on the Boolean hypercube has a unique extension to the entire domain $\mathbb{F}^k$ as a multilinear polynomial.

Definition 2.1 (Multilinear Extension)**.** *The* multilinear extension *(MLE) of* $f : \{0,1\}^k \to \mathbb{F}$ *is the unique multilinear polynomial* $\hat{f} : \mathbb{F}^k \to \mathbb{F}$ *that agrees with* f *on* $\{0,1\}^k$*.*

The multilinear extension can be computed explicitly as follows. For any point $z = (z_1, \ldots, z_k) \in \mathbb{F}^k$:

$$\hat{f}(z) = \sum_{b \in \{0,1\}^k} eq(z, b) \cdot f(b),$$

where eq is equality polynomial defined as:

$$eq(b, z) = \prod_{i=1}^{k} \big(b_i \cdot z_i + (1 - b_i) \cdot (1 - z_i)\big).$$

Throughout this work, for a vector $v \in \mathbb{F}^N$ where $N = 2^k$, we identify v with the function $v : \{0,1\}^k \to \mathbb{F}$ such that $v(b) = v_{\mathsf{int}(b)}$, where $\mathsf{int}(b)$ is the integer with binary representation b. We denote the MLE of the vector by $\hat{v}$.

Lemma 2.2 (Polynomial Identity Lemma)**.** *If* $P : \mathbb{F}^m \to \mathbb{F}$ *is a non-zero total degree* d *polynomial then:*

$$\Pr_{r \in \mathbb{F}^m}[P(r) = 0] < d/|\mathbb{F}|.$$

2.2 Interactive Proofs and IOPs

Interactive Oracle Proofs (IOP) [BCS16, RRR16] are a hybrid between interactive proofs, and *probabilistically checkable proofs*. In this work we consider IOPs for *promise problems*.

Recall that a promise problem consists of two distjoint sets YES and NO. In an IOP for a promise problem $(\mathsf{YES}, \mathsf{NO})$, the goal is for the verifier to accept $x \in \mathsf{YES}$ and reject $x \in \mathsf{NO}$. There is no requirement for instances outside of $\mathsf{YES} \cup \mathsf{NO}$. We assume the verifier is given oracle access to the main input x, and we count the number of queries (in particular, this lets us capture the notion of IOPPs [RRR16]). The prover, who is given x explicitly, is allowed to interact with the verifier over multiple rounds. In each round the prover sends an oracle proof string, and the verifier responds with a random string. At the end of the interaction, the verifier accepts or rejects. For sake of efficiency it will also be convenient to allow the prover to send explicit (i.e., non oracle) messages.

The protocol satisfies *completeness* if for every $x \in \mathsf{YES}$, the prover convinces the verifier with probability 1. The protocol has *soundness error* ϵ if for every $x \in \mathsf{NO}$ and every malicious prover P^* in the IOP model, the probability that the verifier accepts is at most ϵ.

An IOP for a language L is an IOP for the promise problem $(\mathsf{YES}, \mathsf{NO})$, where $\mathsf{YES} = L$ and $\mathsf{NO} = \{0,1\}^* \setminus L$. Similarly, for $\delta > 0$, a δ-IOPP for a language L is an IOP for the promise problem $(\mathsf{YES}, \mathsf{NO})$, where $\mathsf{YES} = L$ and $\mathsf{NO} = \{x : \Delta(x, L) > \delta\}$.

Oracle Aided IOPs. We also consider IOPS in which the verifier has oracle access to some oracle $\mathcal{O}$. We say in this case that the protocol is an $\mathcal{O}$-oracle aided IOP.

2.2.1 Interactive Proofs and the Sumcheck Protocol

We will make use of the celebrated sumcheck protocol [LFKN92], with a linear-time prover [Tha13], as described next:

Lemma 2.3 (The Sumcheck Protocol)**.** *Let $\mathbb{F}$ be a finite field and $m, d \in \mathbb{N}$. There exists a protocol between a prover, who is given as input a description of an m-variate polynomial $Q : \mathbb{F}^m \to \mathbb{F}$ of individual degree d, and a verifier who gets as input $\alpha \in \mathbb{F}$ and either rejects or outputs $(z, v) \in \mathbb{F}^m \times \mathbb{F}$. The protocol satisfies the following properties:*

- ***Complexity:*** *the protocol has m rounds. In each round the prover sends $d+1$ field elements and the verifier responds with a single field element. The verifier performs $O(d \cdot m)$ field operations. The prover runs in time $O((d+1)^m)$. In case Q is a product of d multilinear polynomials, the prover time decreases to $O(d \cdot 2^m)$.*
- ***Completeness:*** *if $\alpha = \sum_{x \in \{0,1\}^m} Q(x)$ then when interacting with the honest prover, the verifier always outputs (z, v) such that $Q(z) = v$.*
- ***Soundness:*** *if $\alpha \neq \sum_{x \in \{0,1\}^m} Q(x)$ then, when interacting with* any *prover, with probability at least $1 - \frac{dm}{|\mathbb{F}|}$, the verifier either rejects or outputs (z, v) such that $Q(z) \neq v$.*

Interactive Proofs for Batching Polynomial Evaluations. We recall an interactive protocol that reduce multiple multilinear evaluation claims into a single one due to [RR24, CBBZ23]. We state the result from [CBBZ23] due to its optimzied prover complexity.

Theorem 2.4 ([CBBZ23, Algorithm 4]). *Let $f : \{0,1\}^k \to \mathbb{F}$ be a function and denote by $\widehat{f}$ its multilinear extension. There exists an interactive protocol that reduces $t > 1$ evaluation claims for $\widehat{f}$ to a single evaluation claim for $\widehat{f}$. The protocol has soundness error $O\left(\frac{k+\log_2(t)}{|\mathbb{F}|}\right)$, prover time $O(t \cdot 2^k)$, verifier time $O(tk)$, communication complexity $O(k+\log(t))$ and $k+\log_2(t)+O(1)$ rounds.*

Furthermore, if only $t' \in [t]$ of the evaluation points are outside the Boolean hypercube, then the prover times is reduced to $O(t' \cdot 2^k)$.[10]

2.3 Coding Theory

Let $\mathbb{F}$ be a finite field. A *code* is an injective function $C : \mathbb{F}^k \to \mathbb{F}^n$. The rate of the code is defined as k/n and its minimal distance is $\min_{x \neq x'} \Delta(C(x), C(x'))$. We say that C is linear if C is a linear function.

We will be interested in proofs of proximity to codes. Actually, following [BCF+25, ACFY25] we will be interested in proofs of proximity to codewords that further satisfy a multilinear evaluation constraint.

Definition 2.5 (Constrained Code). *We define the constrained relation relative to C as:*

$$R_C = \left\{ \big((z,v), c, m\big) \in \mathbb{F}^{\log k} \times \mathbb{F} \times \mathbb{F}^n \times \mathbb{F}^k : c = C(m) \wedge \hat{m}(z) = v \right\},$$

where (z,v) is the explicit input, c is the implicit input and m is the witness.

Interleaved Codes. In this work we utilize *interleaved codes*. For a parameter $t \in \mathbb{N}$, the t-fold interleaving of C, denoted by C^t, consists of all $n \times t$ matrices whose columns belong to C. We sometimes view C^t as a code of blocklength n whose symbols are the entire rows of this matrix.

Thus, $\Delta(M, C^t)$ corresponds to the number of entire rows of M that need to be changed to derive a valid interleaved codeword.

Parity Check Matrices. A code with distance $\gamma > 0$ can be associated with a parity check matrix, defined as follows.

Definition 2.6 (Parity Check Matrix). *Let $\mathbb{F}$ be a finite field. For $n, k \in \mathbb{N}$ and $\gamma \in (0,1)$, an (n,k,γ)-*parity check matrix *is a matrix $H \in \mathbb{F}^{k \times n}$ such that $Hx \neq 0$ for all $x \in \mathbb{F}^n$ with Hamming weight $\mathrm{hw}(x) < \gamma n$.*

2.3.1 Reed-Solomon Code

Let $\mathbb{F}$ be a finite field and fix an evaluation set $L \subseteq \mathbb{F}$. For a parameter $d \leq |\mathbb{F}| - 1$, recall that the $\mathsf{RS}[\mathbb{F}, L, d]$ consists of all functions $f : L \to \mathbb{F}$ that agree with a degree d univariate polynomial. We say that L is smooth if $\mathbb{F}$ is a prime field and L is a multiplicative subgroup, or if $\mathbb{F}$ has characteristic 2 and L is an additive subgroup.

Theorem 2.7 (Constrained RS-IOPP [ACFY25]). *Assume that $|\mathbb{F}| = \Omega(k)$. Then, for every proximity parameter $\delta \in (0, (1-\rho)/2)$, there exists a δ-IOPP for the constrained relation R_{RS} with the following parameters (where λ is the security parameter):*

[10] For the furthermore part, see [CBBZ23, Section 3.8.1].

- ***Soundness Error:*** $2^{-\lambda}$.
- ***Input Query Complexity:*** $\frac{\lambda}{-\log(1-\delta)}$.
- ***Proof Query Complexity:*** $O((\lambda/\delta) \cdot \log\log n)$.
- ***Verifier Time:*** $O(\lambda \cdot \log(n))$.
- ***Prover Time:*** $O(n \log n)$.

2.3.2 Proximity Gaps

We will make use of proximity gaps [RVW13, AHIV23, BKS18] for general linear codes. We use a version that is nowadays referred to as "correlated agreement".

Theorem 2.8 (Correlated Agreement for Linear Codes [BKS18, Theorem 4.1], see also [BCF+25, Corollary 2.4])**.** *Let $C \subseteq \mathbb{F}^n$ be a linear code with distance λ, let $t \in \mathbb{N}$ and let $X \in \mathbb{F}^{n \times t}$ be a matrix such that $\Delta(X, C^t) > \delta$. For any $\eta > 0$ such that $\delta - \eta < \lambda/3$,*

$$\Pr_{r \in \mathbb{F}^t}\left[\Delta(X \cdot r, C) < \delta - \eta\right] \leq \frac{\eta^{-1}}{|\mathbb{F}|}.$$

3 Sketched Coding

Let $k \in \mathbb{N}$ and $\alpha, \gamma \in (0,1)$. Suppose that $H \in \mathbb{F}^{\alpha k \times k}$ is a $(k, \alpha k, \gamma)$-parity check matrix. Let $C : \mathbb{F}^{\alpha k} \to \mathbb{F}^{\alpha k/\rho}$ be a linear error-correcting code for messages of length αk, with rate $\rho \in (0,1)$ and distance $\delta \in (0,1)$.

The sketched code $C_H : \mathbb{F}^k \to \mathbb{F}^{\left(1+\frac{\alpha}{\rho}\right)k}$ is defined as $C_H(x) = (x, C(Hx))$, for every $x \in \mathbb{F}^k$.

Distance Analysis. The sketched codes exhibit a stronger notion of "piecewise distance":

Claim 3.1. *For two distinct codewords $(x, y), (x', y') \in C_H$, either $\Delta(x, x') > \gamma$ or $\Delta(y, y') > \delta$.*

Proof. If $y \neq y'$ then $\Delta(y, y') > \delta$ due to the distance of C. Otherwise we are in the case that $y = y'$ and $x \neq x'$. This means that $Hx = Hx'$ or in other words $H(x - x') = \mathbf{0}$. Thus, as H is a parity check matrix we have that $\Delta(x, x') = \Delta(x - x', 0) > \gamma$. □

Claim 3.1 implies that the sketched code has distance at least:

$$\frac{\min(\gamma k, \delta \alpha k/\rho)}{(1 + (\alpha/\rho))k} = \frac{\min(\gamma, \delta \cdot (\alpha/\rho))}{1 + (\alpha/\rho)},$$

but in this work we will be focusing mostly on the stronger notion of piecewise distance established by Claim 3.1.

4 IOP for Sketched Codes

In this section we construct an IOP for proximity (under the piecewise distance) to a single sketched codeword. Later, in Section 5, we consider the case of interleaved codes.

For $\alpha \in (0,1)$, let $H \in \mathbb{F}^{\alpha k \times k}$ be a $(k, \alpha k, \gamma)$-parity check matrix and $C : \mathbb{F}^{\alpha k} \to \mathbb{F}^{(\alpha/\rho)k}$ be a base code with rate ρ and distance δ. Let $C_H : \mathbb{F}^k \to \mathbb{F}^{(1+\alpha/\rho)k}$ be the sketched code of C as defined in Section 3. That is, $C_H(x) = (x, C(Hx))$.

Let R_C be the constrained relation relative to C (see Definition 2.5). Let $\delta' \in (0, \delta/2)$ and $\gamma' \in (0, \gamma/2)$ be parameters. (Jumping ahead, we will be assuming that R_C has an IOPP relative to distance δ', whereas γ' can be any parameter within the unique decoding radius of the code for which H is a parity check matrix.)

We consider a promise problem $(\mathsf{YES}, \mathsf{NO})$ defined as follows. First, define:

$$\mathsf{YES} = \left\{ \big(C_H(x), z, v\big) \in \mathbb{F}^{(1+(\alpha/\rho))k} \times \mathbb{F}^{\log k} \times \mathbb{F} \;\middle|\; x \in \mathbb{F}^k \text{ and } \hat{x}(z) = v \right\}.$$

We define NO to be the set of tuples $((x,y), z, v) \in \left(\mathbb{F}^k \times \mathbb{F}^{(\alpha/\rho)k}\right) \times \mathbb{F}^{\log k} \times \mathbb{F}$ such that either:

- y is at least δ'-far from C; or
- y is δ'-close to some (unique) codeword $C(u)$ but x is γ'-far from the affine space $\{m \in \mathbb{F}^k : Hm = u\}$; or
- y is δ'-close to some (unique) codeword $C(u)$ and x is γ'-close to a (unique) $m \in \mathbb{F}^k$ such that $Hm = u$, but $\hat{m}(z) \neq v$, where we view m as function $m : \{0,1\}^{\log k} \to \mathbb{F}$.

(In both cases the uniqueness follows from the fact that the relevant vectors are in the unique decoding radius of the respective codes.)

Lemma 4.1. *Suppose that R_C has a δ'-IOPP with input query complexity q_{inp}, proof query complexity q_{prf}, non-oracle communication cc, prover time T_P, verifier time T_V and soundness error $2^{-\lambda}$.*

Then, assuming $|\mathbb{F}| = \Omega(2^\lambda \cdot (k+\lambda))$, the promise problem $(\mathsf{YES}, \mathsf{NO})$ has an $\hat{H}$-oracle aided IOP with the following parameters:

- *Soundness error:* $2^{-\Omega(\lambda)}$.
- *Input Query Complexity: the verifier makes* $\frac{\lambda}{-\log(1-\gamma')}$ *queries to x and q_{inp} queries to y. It fully reads z and v.*
- *Proof Query Complexity:* $O(\lambda \log\log(k)) + q_{prf}$.
- *Non-oracle Communication Complexity:* $O(\log k)$.
- *Prover Time:* $T_P + O(k \cdot \log(k) + \text{sparsity}(H)$ *field operations, where* $\text{sparsity}(H)$ *denotes the number of nonzero entries in H.*
- *Verifier Time:* $T_V + O\left(\log(k) \cdot \frac{\lambda}{\gamma'}\right)$.

4.1 Proof of Lemma 4.1

The input is $(x, y) \in \mathbb{F}^k \times \mathbb{F}^{(\alpha/\rho)k}$ together with $z \in \mathbb{F}^{\log k}$ and $v \in \mathbb{F}$. We simultaneously view x as a function $x : \{0,1\}^{\log k} \to \mathbb{F}$. The protocol proceeds as follows:

1. The prover sends a fresh Reed-Solomon encoding, with rate $\rho' \in (0,1)$, of x, namely $w = \mathsf{RS}_{\rho'}(x)$ of x.
2. The verifier samples a random $r \in \mathbb{F}^{\log(\alpha k)}$ and sends it to the prover. The prover responds with $\sigma = \hat{u}(r)$, where $u = Hx$.
3. The prover and verifier use the sumcheck protocol of Lemma 2.3 to reduce the claim that $\sigma = \sum_{i \in \{0,1\}^{\log k}} \hat{H}(r, i) \cdot x(i)$ to a claim on $\hat{H}(r, s)$ and a claim that $\hat{x}(s) = \tau$, for $s \in \mathbb{F}^{\log(k)}$ and $\tau \in \mathbb{F}$. The claim about $\hat{H}$ is checked directly using the provided oracle.
4. The receiver chooses at random a (multi-)set $I \subseteq \{0,1\}^{\log k}$ of size $t = \frac{\lambda}{-\log(1-\gamma')}$. The receiver reads $x(i)$ from the input for all $i \in I$.
5. The parties run the protocol of Theorem 2.4 to reduce the evaluations claims $\hat{x}(z) = v$, $\hat{x}(s) = \tau$ and $\{x(i)\}_{i \in I}$ to a single claim $\hat{x}(z') = v'$ for some $z' \in \mathbb{F}^{\log k}$ and $v' \in \mathbb{F}$.
6. The parties run the IOPP for constrained RS codes of Theorem 2.7 relative to input w, the evaluation point z' and evaluation v'.
7. The parties run the IOPP for the constrained relation of C relative to y, the evaluation point r and corresponding evaluation σ.
8. The verifier accepts if all of the above procedures accept, otherwise it rejects.

Remark 4.2. *In our actual implementation, the code C is also a Reed-Solomon code and so we can batch the protocols in Steps 6 and 7. Additionally, the underlying IOPP for Reed-Solomon codes [ACFY25, NA25] support parallelizing the proximity check with a sumcheck which yields a further concrete optimization.*

Completeness. Fix an input $\big((x, y), z, v\big)$ where $(x, y) = C_H(x)$, $z \in \mathbb{F}^{\log k}$ and $v \in \mathbb{F}$ so that $\hat{x}(z) = v$, where recall that we view $x : \{0,1\}^{\log k} \to \mathbb{F}$ also as a function. Denote $u = Hx$ and note that $y = C(u)$.

Recall that the prover sends in its first message $w = \mathsf{RS}_{\rho'}(x)$. Let $r \in \mathbb{F}^{\log(\alpha k)}$ be the corresponding verifier message. Since $Hx = u$ it holds that $\sum_{i \in \{0,1\}^{\log k}} \hat{H}(r, i) \cdot x(i) = \hat{u}(r)$.

Hence, the sumcheck protocol invoked in Step 3 is run on a correct claim. Thus, by completeness of the sumcheck protocol, the resulting claim on $\hat{H}$ and $\hat{x}(s) = \tau$ are correct.

Then, in Step 5 the parties invoke the protocol of Theorem 2.4 on a correct claim, and therefore $\hat{x}(z') = v'$.

As such, in Step 6 the prover and verifier invoke the protocol of Theorem 2.7 on a correct claim — that w is an encoding of a message x that is consistent with the claim $\hat{x}(z') = v'$. Similarly, in Step 7 the protocol is invoked on a correct claim — that y is an encoding of a message u such that $\hat{u}(r) = \sigma$.

Soundness. Fix an input $((x,y),z,v) \in \mathsf{NO}$. Recall that $x \in \mathbb{F}^k$, $y \in \mathbb{F}^{(\alpha/\rho)k}$ and $z \in \mathbb{F}$.

Fix a (wlog deterministic) cheating prover strategy. We show that the verifier rejects the claim with high probability. We consider several cases:

Case 1: y is far. Suppose that y is δ'-far from C. Then, in Step 7 we run the underlying IOPP on a NO instance and the verifier rejects with all but $2^{-\lambda}$ probability.

Thus, we may assume that y is close to a (unique) codeword $C(u)$ for some $u \in \mathbb{F}^{\alpha k}$.

Case 2: w is far. Denote the message sent by the prover in the first round by w. Suppose that w is $((1-\rho')/2)$-far from $\mathsf{RS}_{\rho'}$. Then, in Step 6 we run the protocol of Theorem 2.7 on a NO instance and the verifier rejects with all but $O(k/|\mathbb{F}|)$ probability.

Thus, we may also assume that w is close to a (unique) codeword $C(m)$ for some $m \in \mathbb{F}^k$.

Case 3: $u \neq Hm$. Consider now the case that $u \neq Hm$. Then, in Step 2, by Lemma 2.2, with all but $\log(\alpha k)/|\mathbb{F}|$ probability, it holds that $\hat{u}(r) \neq \sum_{i \in \{0,1\}^{\log k}} \hat{H}(r,i) \cdot m(i)$. Assume that such an r was indeed chosen.

Let σ be the value claimed by the prover in Step 2. If $\sigma \neq \hat{u}(r)$ then in Step 7 the claim is false and so the verifier rejects with all but $2^{-\lambda}$ probability. Thus, we may assume that $\sigma = \hat{u}(r)$ and in particular that $\sigma \neq \sum_{i \in \{0,1\}^{\log k}} \hat{H}(r,i) \cdot m(i)$.

Thus, in Step 3 the Sumcheck protocol is invoked on a false claim. Hence, with all but $O(k/|\mathbb{F}|)$ probability, either the claim about $\hat{H}(r,s)$ is false, in which case the verifier immediately rejects (via its oracle query to the multilinear extension of H), or the claim about m is false. That is, $\tau \neq \hat{m}(s)$. But this means that in Step 5 one of the claims is false and so it produces a false claim, with all but probability $O(k+\log(t))/|F|$, which then, in Step 6, makes the verifier reject with all but $2^{-\lambda}$ probability.

Thus, we may assume that $u = Hm$.

Case 4: $\hat{m}(z) \neq v$. Then in Step 5 one of the claims is false and so, with all but probability $O(k+\log(t))/|F|$, it produces a false claim which then, in Step 6, makes the verifier reject with all but $2^{-\lambda}$ probability.

Case 5: x is γ'-far from m. In Step 4 the verifier makes t random samples to x. Thus, with all but $(1-\gamma')^t$ probability, it holds that $x(i) \neq m(i)$ for some $i \in I$. Since $t = -\lambda/\log(1-\gamma/3)$ this probability becomes $2^{-\lambda}$.

Assume that the above event indeed occurs. But this means that in Step 5 one of the claims is false and so, with all but probability $O(k+\log(t))/|F|$, it produces a false claim which then, in Step 6, makes the verifier reject with all but $2^{-\lambda}$ probability.

Case 6: x is γ'-close to m. In this case x is at distance at most γ' from $m \in H^{-1}(u)$ and moreover $\hat{m}(z) = v$, in contradiction to our assumption that $((x,y),z,v) \in \mathsf{NO}$.

Complexity. The query complexity is $t = \frac{\lambda}{-\log(1-\gamma')}$ queries to x in Step 4 and q queries to y in Step 7.

The prover time is dominated by $O(k\log(k))$ for creating w in Step 1 (via the FFT algorithm). For Step 3 the prover first generates the vector $\hat{H}(r,\cdot)$ (which can be done in $O(k)$ times using,

e.g., [Rot24]). Invoking the sumcheck then adds an additional $O(k)$ time (see Lemma 2.3). The protocol of Theorem 2.4 also takes time $O(k)$.

5 IOP for Interleaved Sketched Codes

We consider the same setup as in Section 4. For $\alpha \in (0,1)$, let $H \in \mathbb{F}^{\alpha k \times k}$ be a $(k, \alpha k, \gamma)$-parity-check matrix and $C : \mathbb{F}^{\alpha k} \to \mathbb{F}^{(\alpha/\rho)k}$ be a base code with rate ρ and distance δ. Let $C_H : \mathbb{F}^k \to \mathbb{F}^{(1+\alpha/\rho)k}$ be the sketched code of C defined as $C_H(x) = (x, C(Hx))$.

For an interleaving parameter $t \in \mathbb{N}$, which is a power of two, and proximity error $\eta > 0$, we consider the interleaved promise problem $(\mathsf{YES}_t, \mathsf{NO}_t)$, defined as follows.

- The YES_t instances are defined as (X, Y, z, v), where $X \in \mathbb{F}^{k \times t}$, $Y \in \mathbb{F}^{((\alpha/\rho)\cdot k)\times t}$, $z \in \mathbb{F}^{\log(k)+\log(t)}$ and $v \in \mathbb{F}$ such that $Y = C^t(HX)$ and $\hat{X}(z) = v$.
- The NO_t instances are defined as (X, Y, z, v) such that one of the following conditions holds:
  - Y is $(\delta/3 + \eta)$-far from the interleaved code C^t; or
  - Y is $(\delta/3 + \eta)$-close to a (unique) interleaved codeword $C^t(U)$ for $U \in \mathbb{F}^{\alpha k \times t}$ but X is $(\gamma/3 + \eta)$-far from the affine space $\{M \in \mathbb{F}^{k\times t} : HM = U\}$; or
  - Y is $(\delta/3 + \eta)$-close to a (unique) $C^t(U)$ and X is $(\gamma/3 + \eta)$-close to a (unique) M with $HM = U$, but $\hat{M}(z) \neq v$.

  (In the latter two cases the codewords are unique as they are within the unique decoding radius of the respective codes.)

Lemma 5.1. *Suppose that R_C has a δ'-IOPP with input query complexity q_{inp}, proof query complexity q_{prf}, non-oracle communication cc, prover time T_P, verifier time T_V and soundness error $2^{-\lambda}$.*

Then, for every $t \in \mathbb{N}$, assuming $|\mathbb{F}| = \Omega(2^\lambda \cdot (k + \lambda))$, the promise problem $(\mathsf{YES}_t, \mathsf{NO}_t)$ has an $\hat{H}$-oracle aided IOP, with the following parameters:

- *Soundness error:* $2^{-\Omega(\lambda)}$.
- *Query Complexity: the verifier makes* $\frac{\lambda}{-\log(1-\gamma/3)}$ *queries to the rows of X and q_{inp} queries to the rows of Y. It fully reads z and v.*
- *Proof Query Complexity:* $O(\lambda \log\log(k)) + q_{prf}$.
- *Non-oracle Communication Complexity:* $O(t + \log k)$.
- *Prover Time:* $2k \cdot t$ *field additions and multiplications and an additional* $T_P + O(k \log(k)) +$ $\mathrm{sparsity}(H)$ *field operations, where* $\mathrm{sparsity}(H)$ *denotes the number of nonzero entries in H.*
- *Verifier Time:* $T_V + O\left(t \cdot \log(k) \cdot \frac{\lambda}{\gamma}\right)$.

5.1 Proof of Lemma 5.1

The protocol on input (X, Y, z, v) proceeds as follows:

1. Parse z as $z = (z_1, z_2) \in \mathbb{F}^{\log k} \times \mathbb{F}^{\log t}$.
2. The prover generates and sends (as an explicit message) the function $g : \{0,1\}^{\log t} \to \mathbb{F}$ defined as $g(b) = \hat{X}(z_1, b)$, for $b \in \{0,1\}^{\log t}$.
3. The verifier checks that $\hat{g}(z_2) = v$.
4. The verifier chooses a random $r \in \mathbb{F}^t$ and sends it to the prover.[11]
5. The prover and verifier run the IOP of Lemma 4.1 on the input $(x, y, z_1, \langle r, g\rangle)$, where $x = Xr$ and $y = Yr$. The protocol is invoked relative to parameter $\delta' = \delta/3$ and $\gamma' = \gamma/3$. Each query to x (resp. y) is emulated via a single query to the corresponding full row of X (resp., Y).

Completeness. Fix an input $((X, Y), z, v)$ where $(X, Y) = C^t_H(X)$, $z \in \mathbb{F}^{\log k + \log t}$ and $v \in \mathbb{F}$ so that $\hat{X}(z) = v$, where recall that we view X as a function $X : \{0,1\}^{\log k + \log t} \to \mathbb{F}$. Let $U = HX$ and note that $Y = C^t(U)$.

Recall that the prover sends in its first message $g = \hat{X}(z_1, *)$. Note that the verifier's check in Step 3 passes since $\hat{g}(z_2) = \hat{X}(z_1, z_2) = v$.

Let $r \in \mathbb{F}^t$ be the random vector chosen by the verifier in Step 4. Observe that, by linearity, $y = Yr = C^t(HX)r = C^t(HXr) = C^t(Hx)$ and that:

$$\hat{x}(z_1) = \widehat{Xr}(z_1) = \langle r, \hat{X}(z_1, *)\rangle = \langle r, g\rangle$$

and so (X, Y, z, v) is a YES instance for the IOPP of Lemma 4.1 and so the verifier accepts.

Soundness. Fix a NO instance (X, Y, z, v). We consider several cases:

Case 1: Y is $(\delta/3 + \eta)$-far from C^t. By Theorem 2.8, we have that $y = Yr$ is $(\delta/3)$-far from C with all but $\frac{\eta^{-1}}{|\mathbb{F}|}$ probability. Assume that this is indeed the case. But this means that the input to the IOPP of in Step 5 is a NO instance and so the verifier rejects with all but $2^{-\lambda}$ probability.

Thus, we may assume that Y is $(\delta/3)$-close to a unique codeword $C^t(U)$, for some $U \in \mathbb{F}^{\alpha k \times t}$. By linearity this means that $y = Yr$ is also $(\delta/3)$-close to $C(u)$, for $u = Ur$.

Case 2: X is $(\gamma/3 + \eta)$-far from the affine space. Next, suppose that X is $(\gamma/3 + \eta)$-far from any M for which $H \cdot M = U$. Thus, $X - H^{-1}(U)$ is $(\gamma/3 + \eta)$-far from the linear space $H^{-1}(\mathbf{0})$. Note that the latter space is exactly the interleaving of the code whose parity check is H, and so has distance at least γ. Applying Theorem 2.8 with proximity parameter $\gamma/3 + \eta$ and slack η, this means that with all but $\frac{\eta^{-1}}{|\mathbb{F}|}$ probability, it holds that $(X - H^{-1}(U)) \cdot r$ is at least $(\gamma/3)$-far from $H^{-1}(\mathbf{0})$. In other words, $x = Xr$ is at least $(\gamma/3)$-far from:

$$H^{-1}(\mathbf{0}) + H^{-1}(U) \cdot r = H^{-1}(\mathbf{0}) + H^{-1}(Ur) = H^{-1}(u),$$

[11] One could partially derandomize r by using a suitable small bias generator as shown by Diamond and Posen [DP24].

where $u = Ur$ and both equalities follow from linearity.

Hence, the instance $(x, y, z_1, \langle g, r\rangle)$ on which we run the protocol from Lemma 4.1 in Step 5 is a NO instance and the verifier rejects with all but $2^{-\lambda}$ probability.

Thus, we may assume that X is $(\gamma/3+\eta)$-close to some M such that $HM = U$. This implies, in particular, that $x = Xr$ is $(\gamma/3+\eta)$-close to $m = Mr$ and that $Hm = u$, where $u = Ur$.

Case 3: $\hat{g}(z_2) \neq v$**.** In this case the verifier immediately rejects in Step 3. Thus, we may assume that $\hat{g}(z_2) = v$.

Case 4: $\hat{M}(z_1, z_2) \neq v$**.** In this case, note that $w = \hat{M}(z_1, *)$ and g are distinct vectors (they disagree at z_2). Thus, with all but $1/|\mathbb{F}|$ probability, it holds that $\langle w, r\rangle \neq \langle g, r\rangle$. Assume that this is indeed the case.

Thus, we know that y is $\delta/3$-close to $C(u)$ and that x is $(\gamma/3+\eta)$-close to m such that $HM = u$ but $\hat{m}(z_1) = \widehat{Mr}(z_1) = \hat{M}(z_1, *) \cdot r = \langle w, r\rangle \neq \langle g, r\rangle$. But this means that in Step 5 we run the protocol of Lemma 4.1 on a NO instance, and the verifier rejects with all but $2^{-\lambda}$ probability.

Complexity. The query complexity follows from the query complexity of Lemma 4.1. Prover and verifier time also follow directly from the protocol in combination with those of Lemma 4.1.

6 Random LDPCs

In this section we use prior work from the coding theory literature to give high probability guarantees on the distance of LDPC codes. First, in Section 6.1 we consider the case of binary codes and then, in Section 6.2 we consider the case of larger alphabets.

6.1 Random LDPCs over Bits

Gallager [Gal62] (see [Gal63]) introduced random low density parity check codes and analyzed their properties. We start by recalling his result.

The Gallager ensemble of (n, j, k) low-density parity-check (LDPC) codes is defined by a parity-check matrix H of dimension $m \times n$, where $m = n \cdot (j/k)$, constructed as the vertical concatenation of j submatrices $H_1, H_2, \ldots, H_j$, each of size $(n/k) \times n$. The first submatrix H_1 is fixed deterministically such that its r-th row contains ones in the columns $(r-1)k+1$ through rk and zeros elsewhere, ensuring a row weight of k and a column weight of 1 within the strip. For each subsequent strip $i \in \{2, \ldots, j\}$, the submatrix H_i is generated by applying a random column permutation π_i to the columns of H_1.

Theorem 6.1 ([Gal63, Theorem 2.4])**.** *The minimal distance of the Gallager ensemble of (n, j, k) codes has minimal distance at least δ with all but probability:*

$$\frac{k-1}{2n^{j-2}} + o\left(\frac{1}{n^{j-2}}\right) + nC_j(\delta, n) \exp\left[-nB_{j,k}(\delta)\right],$$

where the functions $C(\delta, n)$ and $B_{jk}(\delta)$ are defined as:

$$C_j(\delta, n) = \left(2\pi n\delta(1-\delta)\right)^{\frac{j-1}{2}} \exp\left(\frac{j-1}{12n\delta(1-\delta)}\right)$$
$$B_{j,k}(\delta) = (j-1)H(\delta) - \frac{j}{k}\Big(\mu(s) + (k-1)\ln 2\Big) + js\delta$$

with

$$H(\delta) = -\delta\ln\delta - (1-\delta)\ln(1-\delta)$$

and the function $\mu(s)$ is given by:

$$\mu(s) = \ln\left(2^{-k}\Big((1+e^s)^k + (1-e^s)^k\Big)\right).$$

and the parameter s can be arbitrary but the optimal choice is the unique solution to the equation $\mu'(s) = \delta \cdot k$ (where μ' is the derivative of μ).

Remark 6.2. *To make the error probability small, we first and foremost need to ensure that the parameter $B_{j,k}$ is positive. If is indeed positive, the last term vanishes and the error becomes dominated by the $\frac{k-1}{2n^{j-2}}$ factor.*

Gallager suggests to expurgate *the code, by checking that all weight 2 messages are not mapped to the all-zero syndrome. This eliminates the first error term.*

6.1.1 Concrete Analysis

We analyze Gallager's code for column weight $j = 8$, row weight $k = 64$ and distance 0.013.

Exact Expression for the Exponent $B_{8,64}(0.013)$. The derivative $\mu'(s)$ is given by:

$$\mu'(s) = \frac{ke^s\left[(1+e^s)^{k-1} - (1-e^s)^{k-1}\right]}{(1+e^s)^k + (1-e^s)^k}.$$

We need to find s such that $\mu'(s) = 0.013 \cdot 64$. Solving numerically, using a Python script [GNR26], we find that the solution to the equation $\mu'(s) = \delta \cdot k$ is $s \approx -4.087290$. Plugging that back into the definition of B, we get that $B \approx 0.000213$ is positive and so the exponent vanishes.

Remark 6.3. *We remark that the correctness of the numerical estimation for s is not an assumption for our result. This is because Gallager's result holds for any value of s and we use the numerical analysis merely to derive a (close to optimal) choice for s.*

Exact Expression for the Coefficient $C_8(0.01, n)$. Substituting the parameters into the coefficient function:

$$C_8(0.013, n) = \left(2\pi n(0.013)(0.987)\right)^{3.5} \exp\left(\frac{7}{12n(0.013)(0.987)}\right) = O(n^{3.5}).$$

Conclusion. By Theorem 6.1, and using the analysis above, the probability that the code fails to have distance at least 0.013 is at most $O(1/n^6)$.

6.1.2 Theoretical Analysis

We show that Gallager's result implies the following:

Theorem 6.4. *For any $\alpha > 0$ there exists a constant $\gamma > 0$ such that a random matrix from the $(n, 3, 3/\alpha)$ Gallager ensemble is an $(n, \alpha n, \gamma)$-parity check matrix with all but $O(1/n)$ probability.*

Let $\alpha > 0$. To prove Theorem 6.4 we instantiate Theorem 6.1 using $j = 3$ and $k = 3/\alpha$. to bound the error probability we need to bound the error terms $B_{3,k}$ and C_3.

Bounding $B_{3,k}$. Recall that s in Theorem 6.1 can be set arbitrarily and so $c = \frac{3}{k}\Big(\mu(s) + (k-1)\ln 2\Big)$ and s are fixed constants independent of δ. Thus, it suffices to find $\delta > 0$ such that

$$2H(\delta) + 3s\delta > c.$$

Such a δ exists since the entropy function is asymptotically larger than any fixed linear function sufficiently close to 0.

Bounding C_3. Recall that $C_3(\delta, n) = 2\pi n\delta \cdot (1-\delta) \cdot \exp\left(\frac{1}{6n\delta\cdot(1-\delta)}\right)$. For any fixed $\delta > 0$ this term is $O(n)$.

Conclusion. Overall, we get that the error probability from Theorem 6.1 is

$$\frac{k-1}{2n} + o(1/n) + O(n^2 \cdot \exp^{-nB_{j,k}(\delta)}) = O(1/n).$$

6.2 Random LDPC over Large Fields

We use an extension of Gallager's result to larger fields, due to Yang *et al.* [YHC+11] (improving over [BB04, CF08]), where the parity check matrix has random coefficients over the large field (Gallager's original work also consideres the large field case, but when the parity check matrix still has Boolean values).

Let $j, k, q \in \mathbb{N}$. The random (j, k, q)-LDPC code ensemble $\mathcal{C}^{(n)}_{q,j,k}$ over the field $\mathbb{F}_q$ is defined through a random bipartite graph with n left nodes called *variable nodes* of degree j and nj/k right notes, called *check nodes*, of degree k. We refer to the jn outgoing edges of the variable nodes as variable sockets and the $(nj/k) \cdot k = nj$ incoming edges of the check nodes as check sockets. Thus, the edges induce a bijection between the variable sockets and the check sockets.

The connections between these two sets of sockets are determined by a uniformly random bijection $\pi : [n] \times [j] \to [nj/k] \times [k]$. Additionally, each check socket $(b, \beta) \in [nj/k] \times [k]$ is assigned an independent (non-zero) random weight $w_{b,\beta} \in \mathbb{F}_q^\times$.

Given a vector $v \in \mathbb{F}_q^n$, we assign values to check nodes where the b-th node has value:

$$\sum_{a\in[n],\alpha\in[j],\beta\in[k] \text{ s.t. } \pi(a,\alpha)=(b,\beta)} v_a \cdot w_{b,\beta}.$$

The code $\mathcal{C}^{(n)}_{q,j,k}$ consists of all vectors $v \in \mathbb{F}_q^n$ for which all check nodes have value 0.

Theorem 6.5 ([YHC+11, Theorem 6.2]). *Let $q \geq 2$, $k \geq j \geq 3$ such that j is even. Then, the minimal distance of the ensemble $\mathcal{C}_{q,j,k}^{(n)}$ is at least $\delta \in (0, 1 - 1/q)$ with all but probability:*

$$O\left(n^{-\lceil (j-2)/2 \rceil} + n^{\frac{3}{2}} \cdot e^{n\omega_{q,j,k}(\delta)})\right)$$

where $\omega_{q,j,k}(x)$ is defined as:

$$\omega_{q,j,k}(x) \triangleq H_q(x) + \frac{j}{k}[\delta_{q,k}(x) - \ln q],$$

with

$$\delta_{q,k}(x) \triangleq \inf_{\hat{x} \in (0,1)} \{k \cdot D(x||\hat{x}) + \rho_{q,k}(\hat{x})\},$$

$$\rho_{q,k}(x) \triangleq \ln \left[1 + (q-1) \cdot \left(1 - \frac{qx}{q-1}\right)^k\right],$$

and where $H_q(x)$ is the q-ary entropy function $H_q(x) \triangleq x \ln \frac{1}{x} + (1-x) \ln \frac{1}{1-x} + x \ln(q-1)$, and $D(x||\hat{x})$ is the information divergence $D(x||\hat{x}) \triangleq x \ln \frac{x}{\hat{x}} + (1-x) \ln \frac{1-x}{1-\hat{x}}$.

Concrete Instantiation. We consider a concrete instantiation when $q = 2^{32}$, $j = 16$ and $k = 128$. To calculate ω, we use the fact, shown in [YHC+11, Theorem 5.6], that ω has a unique root at $x_0 \in (0, 1)$. Furthermore, as shown in the same theorem the function is continuous on $(0, 1)$ and is negative on $(0, x_0)$ and positive on $(x_0, 1)$. We can set δ to therefore be very slightly smaller than x_0 to get that the error probability in Theorem 6.5 is $O(n^{-7})$, details follow.

Using a script, available at [GNR26], we find that $x_0 \approx 0.094114$. As in Remark 6.3 we do not rely on the correctness of the script and merely use the candidate value to show that the distance is at least $\delta = 0.0941$ with high probability.

Let us first compute $H_q(\delta)$:

$$H_q(\delta) = \delta \ln \frac{1}{\delta} + (1-\delta) \ln \frac{1}{1-\delta} + \delta \ln(q-1) \leq 2.39912809.$$

Consider $\hat{x} = 0.16131360844439377$. Since δ is computed as an infimum, we have that $\delta_{q,k}(x_0) \geq k \cdot D(x_0||\hat{x}) + \rho_{q,k}(\hat{x})$. We have that:

$$D(x_0||\hat{x}) \leq 0.01911026$$

and

$$\rho_{q,k}(\hat{x}) \leq 0.53222.$$

Thus, we have that $\delta_{q,k}(x_0) \leq 2.97833328$. Plugging that back into the formula for ω, we get that $\omega_{q,j,k} \leq -0.00116897$. Since the bound for ω is negative, we get that the code has distance at least δ with high probability.

7 Instantiations

7.1 A Fast Code: Proof of Theorem 1.2

To prove Theorem 1.2, for a given desired rate parameter $r > 0$ and $\varepsilon > 0$, we need show that there exists $\delta(r, \varepsilon) > 0$ and a family of codes that are computable by circuits of size $(3 + \varepsilon) \cdot k$, such that a random code from the family has distance δ with all but $O(1/n)$ probability.

Let C be a linear-time error correcting code with constant rate $\rho > 0$, constant distance $\delta > 0$, and let c be a constant such that its encoding can be done in time $c \cdot k$ (e.g., the linear-time encodable code due to [Spi96]). Instantiating the sketched code using C and the parity check matrix distribution of Theorem 6.4, using $\alpha = \min\left(\varepsilon, \frac{1}{1+r}\right) \cdot c^{-1}$ we get a code $C_H : \{0,1\}^k \to \{0,1\}^{(1+\alpha/\rho)k}$ that is encodable in time $3k + c \cdot (\alpha k) \leq (3 + \varepsilon)k$. Note that $c \geq 1/\rho$ and $\alpha \leq \frac{1}{1+r}$ and so the rate of the code is at least r.

By the above analysis, the code has distance $\frac{\min(\gamma, \delta \cdot (\alpha/\rho))}{1+(\alpha/\rho)}$, which is a positive constant.

7.2 An Asymptotically Efficient PCS: Proof Sketch of Theorem 1.1

To construct our MLPCS, we utilize the linear-time encodable code from [BFRW25], which we denote by $C : \{0,1\}^{\alpha k} \to \{0,1\}^{(\alpha/\rho)k}$. (We remark that this code is itself a tensor encoding of a linear-time encodable code, such as the RAA ensemble from [BCF+25].)

We utilize the fact, shown in [BFRW25], that there exists an IOPP for the constrained code C with a linear-time prover and $O(\lambda)$ queries, where λ is the security parameter (for the asymptotic result, one could similarly use the code from [BMMS25b]).

We then instantiate Lemma 5.1 using an interleaving parameter t and a random LDPC matrix H from Theorem 6.1. This gives us an $\hat{H}$-oracle aided IOP for the promise problem $(\mathsf{YES}_t, \mathsf{NO}_t)$ defined in Section 5 with:

- Soundness error: $2^{-\Omega(\lambda)}$.
- Query Complexity: the verifier makes a total of $O(\lambda)$ queries to the rows of the input.
- Proof Query Complexity: $O(\lambda \log\log(k))$.
- Non-oracle Communication Complexity: $O(t + \log k)$.
- Prover Time: $O(k \cdot (t + \log(k)))$.
- Verifier Time: $O(\lambda \cdot t \cdot \log(k))$.

Setting $t = \log(N)$ and $k = N/\log(N)$ we get that the IOP evaluation proof requires $(1+o(1))N$ field additions and multiplications. The input query complexity is $O(\lambda)$, the proof oracle complexity is $O(\lambda \cdot \log\log N)$, the non-oracle communication complexity is $O(\log N)$ and the verifier time is $\lambda \cdot \tilde{O}(\log N)$.

We obtain an MLPCS as follows. To commit to a message $M \in \mathbb{F}^{k \times t}$, we send its encoding under the interleaved code C_H^t. The evaluation proof simply runs the IOP. To see that this is indeed binding, consider an adversarially generated commitment (X, Y). If $((X, Y), z, v)$ is a NO instance, then the verifier rejects with high probability. If it is not a NO instance, then (X, Y) can be uniquely decoded to a message M that is determined already at the time of commitment.

The MLPCS is relative to the oracle $\hat{H}$, we convert this into a preprocessing MLPCS by utilizing a sparse polynomial commitment, such as Spark [Set20].

As the final step, we then covert the IOPCS into a MLPCS in the random oracle model using the standard transformation [Kil92, Mic00, BCS16].

7.3 The Concrete Instantiation

As discussed in Section 1.2, in our concrete instantiation we use a different parameter regime, for two sets of parameters. For **bolt-max**, we use a parity check matrix H from Theorem 6.1 with column sparsity 8 and compression factor $\alpha = 1/8$, where elements are bits. The numerical instantiation of Theorem 6.1, shown in Section 6.1.1 yields that the resulting code has distance at least $\gamma = 0.013$ with all but $O(1/k)$ probability. For **bolt-min**, we use a parity check matrix H with column sparsity 16 and compression factor $\alpha = 1/8$, where elements are taken from $\mathrm{GF}(2^{32})$. The resulting code has distance at least $\gamma = 0.096$, based on the analysis in [YHC+11].

We work over the field $\mathrm{GF}(2^{32})$, a binary extension field. As our base code for the code sketch, we use the Reed-Solomon code of rate $1/2$ over this field. We instantiate Lemma 5.1 with these parameters to obtain an IOP for the interleaving sketched code, while using Ligerito [NA25], in the unique decoding radius, as our underlying proximity proof.

This yields an $\hat{H}$-oracle aided IOPP for the promise problem $(\mathsf{YES}_t, \mathsf{NO}_t)$ defined in Section 5 with query complexity $\lambda \cdot \left(\frac{1}{-\log(1-\gamma/3)} + \frac{1}{-\log(1-1/4)}\right)$ to the rows of the input matrix.

The commitment time is proportional to the encoding time of the interleaved sketched code with the above parameters and an additional Merkle tree computation of size $(1 + \alpha/\rho) \cdot k = 1.25k$ field elements.

8 Benchmarks and Evaluation

We evaluate the performance of Bolt compared to other popular multilinear PCS's. We consider committing to a 30-variate multilinear over the field $\mathbb{F} = \mathrm{GF}(2^{32})$ (representing a total of 4GBs of data). We represent the field using the irreducible polynomial $x^{32} + x^{15} + x^9 + x^7 + x^4 + x^3 + 1$. We target 100 bits of security and so use the degree 4 extension of $\mathbb{F}$ (a 128-bit field) for the random linear combinations.

The benchmarks were performed using an Apple Mac mini with an M4 Pro processor with 14 CPU cores, 20 GPU cores and 64 GBs of memory. All implementations incorporate vectorization (aka SIMD), multi-threading, cache optimizations, pre-fetching and metal GPU acceleration where applicable. The implementations utilize the PMULL instruction for "carryless multiplication" to efficiently implement binary field multiplication. We utilize the SHA-256 hash function, and utilize hardware acceleration for that hash that is available on Macs. The implementation is available at [GNR26].

We describe in detail how we evaluated each of the systems.

Bolt. We report benchmarks for two variants of Bolt. The first variant is optimized for prover speed. Hence, it uses an LDPC with coefficients in $\mathrm{GF}(2)$ and therefore avoids multiplications during the LDPC encoding. This makes it significantly faster, at the cost of a smaller distance and consequently larger proofs. In this variant we use an LDPC with "locality" $j = 8$. We refer to this variant as *Bolt-max*.

The second variant is optimized for smaller proofs. Here we use an LDPC with coefficients over a larger field as described in Section 6.2. This implementation is somewhat slower, since encoding requires field multiplications, but it yields significantly smaller proofs. In this variant we use a locality of $j = 16$. We refer to this variant as *Bolt-min*.

The systematic, high-rate structure of the code is utilized for efficient encoding. Apple M-series and A-series chips provide distinct CPU and GPU compute units with a shared memory architecture, as well as hardware acceleration for SHA-256 on the CPU. In the SHA-256 setting, we exploit this by overlapping hashing and encoding across these units and assign encoding to the CPU and hashing to the GPU, and then building the tree on the CPU.

The evaluation proof of Bolt, described in Section 5, includes Reed-Solomon proximity proofs for two encoded oracles: one corresponding to a re-encoding of the random linear combination of the underlying message, and the other for the encoding of the syndromes. It also includes one sumcheck to batch the spot check claims and then one to reduce the lookup to a sparse PCS evaluation.

In the amortized setting, the sparse PCS evaluation cost becomes negligible. In the non-amortized setting we run the Mulperm protocol, due to Bünz *et al.* [BCD25]. We report the performance for both versions below.

Remark 8.1 (Mulperm Cost)**.** *At the current time, an implementation of the Mulperm protocol [BCD25] is not yet available. That protocol is for our non-amortized evaluation proof and so we estimate the cost as follows.*

The Mulperm protocol is dominated by a sumcheck for a product of d multilinears each on n variables, where $d = 1 + \lceil \sqrt{\log n} \rceil$ and where n depends on the Bolt instantiation: $n = 26$ for Bolt-max *and $n = 27$ for* Bolt-min*. Using standard sumcheck optimization techniques, the cost is dominated by $d \cdot (d-1) \cdot 2^n$ multiplications.[12] Plugging in our concrete parameters, together with the measured performance of one inner-product sumcheck implementation, we estimate the running time of Mulperm, for our setting of parameters, to be* 1.5 *seconds in our benchmarks.*

Blaze [BCF+25]. A reference implementation of the Blaze protocol is available at `https://github.com/hadasz/plonkish_basefold` but is not compatible with our hardware (it is optimized for the Intel x86 architecture, whereas we use an Apple M4). Thus, we re-implemented and optimized the commitment phase of Blaze (which consists of an interleaved RAA encoding).

As for the Blaze evaluation proof, to make it an apples-to-apples comparison, we consider an optimized version that incorporates new sub-protocols that were not available at the time that Blaze was published. Recall that the evaluation proof of Blaze consists of 2 sumchecks, 2 permutation checks and sending 5 intermediate oracles: one corresponding to a re-encoding of the message of the random linear combination and four encodings for intermediate steps of the RAA encoding. The original Blaze paper uses Basefold [ZCF24] to commit to those oracles, we replace it with the more efficient Ligerito protocol, using an RS code of rate 1/2. Further, the original Blaze protocol uses Spark [Set20] for the permutation check. As Spark involves sending multiple additional oracles, we replace it with recent and highly efficient Mulperm protocol [BCD25], for which we estimate the cost via Remark 8.1.

Ligerito [NA25]. To benchmark Ligerito we ported the Reed-Solomon implementation over binary fields from `https://github.com/bcc-research/Ligerito.jl`. We remark that the prover

[12] The cost can be reduced to $\tilde{O}(d) \cdot 2^n$ via the FFT (see, e.g., [ARR25]). However, for small values of d this does not seem attractive.

cost of Ligerito is fairly similar to that of WHIR [ACFY25], but the implementation supports binary fields. Our implementation has been optimized significantly.

Brakedown [GLS+23]. A reference implementation is available at `https://github.com/hadasz/plonkish_basefold` as a fork of the original private implementation. However it seems under-optimized and does not give competitive numbers. Thus, we have re-implemented Brakedown while also taking into account an improved analysis due to Haböck [Hab23].

We include numbers for two variants of Brakedown that were reported in [GLS+23]. The first optimizes for prover speed with distance 0.02 and second optimizes for proof size, with distance 0.07.

The results for all benchmarks are reported in Table 1.

	Commitment Time (ms)	**Eval Proof Time (ms)**	**Proof size**
Ligerito	3590	359	0.6MB
Blaze	1510	4929 *	3.4MB
Brakedown $d = 0.02$	1110	132	57.8MB
Brakedown $d = 0.07$	1480	129	29.7MB
Bolt-max	**840**	**162 (1696* unamortized)**	**12.92MB**
Bolt-min	**1250**	**157 (3139* unamortized)**	**2.09MB**

Table 1: Multilinear PCS with 30-variables over GF(2^{32}) (a total 4GB of data). For the evaluation proof we include the cost of generating the random linear combination of the underlying message. Numbers marked with $*$ are based on estimated cost for Mulperm, using sumcheck benchmarks from [Irr25]. For Ligerito, we only count the significant work from the first folding step.

References

[ACFY24] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. STIR: Reed-Solomon proximity testing with fewer queries. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part X*, volume 14929 of *Lecture Notes in Computer Science*, pages 380–413. Springer, 2024. 3

[ACFY25] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. WHIR: Reed-Solomon proximity testing with super-fast verification. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology - EUROCRYPT 2025 - 44th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Madrid, Spain, May 4-8, 2025, Proceedings, Part IV*, volume 15604 of *Lecture Notes in Computer Science*, pages 214–243. Springer, 2025. 3, 4, 10, 14, 17, 28

[AHIV23] Scott Ames, Carmit Hazay, Yuval Ishai, and Muthuramakrishnan Venkitasubramaniam. Ligero: lightweight sublinear arguments without a trusted setup. *Des. Codes Cryptogr.*, 91(11):3379–3424, 2023. 3, 6, 7, 9, 15

[ARR25] Noor Athamnah, Noga Ron-Zewi, and Ron D. Rothblum. Linear prover IOPs in log star rounds. In Benny Applebaum and Huijia Rachel Lin, editors, *Theory of Cryptography*

- *23rd International Conference, TCC 2025, Aarhus, Denmark, December 1-5, 2025, Proceedings, Part I*, volume 16268 of *Lecture Notes in Computer Science*, pages 335–368. Springer, 2025. 3, 27

[BB04] Amir Bennatan and David Burshtein. On the application of LDPC codes to arbitrary discrete-memoryless channels. *IEEE Trans. Inf. Theory*, 50(3):417–438, 2004. 23

[BBHR18] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Fast Reed-Solomon interactive oracle proofs of proximity. In Ioannis Chatzigiannakis, Christos Kaklamanis, Dániel Marx, and Donald Sannella, editors, *45th International Colloquium on Automata, Languages, and Programming, ICALP 2018, Prague, Czech Republic, July 9-13, 2018*, volume 107 of *LIPIcs*, pages 14:1–14:17. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2018. 3

[BCD25] Benedikt Bünz, Jessica Chen, and Zachary DeStefano. Linear*-time permutation check. *IACR Cryptol. ePrint Arch.*, page 1850, 2025. 4, 10, 11, 27

[BCF+25] Martijn Brehm, Binyi Chen, Ben Fisch, Nicolas Resch, Ron D. Rothblum, and Hadas Zeilberger. Blaze: Fast SNARKs from interleaved RAA codes. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology - EUROCRYPT 2025 - 44th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Madrid, Spain, May 4-8, 2025, Proceedings, Part IV*, volume 15604 of *Lecture Notes in Computer Science*, pages 123–152. Springer, 2025. 3, 4, 5, 6, 8, 10, 14, 15, 25, 27

[BCG+17] Jonathan Bootle, Andrea Cerulli, Essam Ghadafi, Jens Groth, Mohammad Hajiabadi, and Sune K. Jakobsen. Linear-time zero-knowledge proofs for arithmetic circuit satisfiability. In *Proceedings of the 23rd International Conference on the Theory and Applications of Cryptology and Information Security*, ASIACRYPT '17, pages 336–365, 2017. 3

[BCG20] Jonathan Bootle, Alessandro Chiesa, and Jens Groth. Linear-time arguments with sublinear verification from tensor codes. In *Proceedings of the 18th Theory of Cryptography Conference*, TCC '20, pages 19–46, 2020. 3

[BCH+25] Eli Ben-Sasson, Dan Carmon, Ulrich Haböck, Swastik Kopparty, and Shubhangi Saraf. On proximity gaps for Reed-Solomon codes. *Electron. Colloquium Comput. Complex.*, TR25-169, 2025. 11

[BCI+23] Eli Ben-Sasson, Dan Carmon, Yuval Ishai, Swastik Kopparty, and Shubhangi Saraf. Proximity gaps for Reed-Solomon codes. *J. ACM*, 70(5):31:1–31:57, 2023. 11

[BCS16] Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner. Interactive oracle proofs. In Martin Hirt and Adam D. Smith, editors, *Theory of Cryptography - 14th International Conference, TCC 2016-B, Beijing, China, October 31 - November 3, 2016, Proceedings, Part II*, volume 9986 of *Lecture Notes in Computer Science*, pages 31–60, 2016. 13, 26

[BFRW25] Benedikt Bünz, Giacomo Fenzi, Ron D. Rothblum, and William Wang. Tensorswitch: Nearly optimal polynomial commitments from tensor codes. *IACR Cryptol. ePrint Arch.*, page 2065, 2025. 3, 4, 6, 25

[BHR05] Eli Ben-Sasson, Prahladh Harsha, and Sofya Raskhodnikova. Some 3CNF properties are hard to test. *SIAM J. Comput.*, 35(1):1–21, 2005. 10

[BKS18] Eli Ben-Sasson, Swastik Kopparty, and Shubhangi Saraf. Worst-case to average case reductions for the distance to a code. In Rocco A. Servedio, editor, *33rd Computational Complexity Conference, CCC 2018, San Diego, CA, USA, June 22-24, 2018*, volume 102 of *LIPIcs*, pages 24:1–24:23. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2018. 7, 15

[BMMS25a] Anubhav Baweja, Pratyush Mishra, Tushar Mopuri, and Matan Shtepel. FICS and FACS: fast IOPPs and accumulation via code-switching. *IACR Cryptol. ePrint Arch.*, page 737, 2025. 3, 4, 8

[BMMS25b] Anubhav Baweja, Pratyush Mishra, Tushar Mopuri, and Matan Shtepel. Query-optimal IOPPs for linear-time encodable codes. *IACR Cryptol. ePrint Arch.*, page 1588, 2025. 3, 4, 25

[CBBZ23] Binyi Chen, Benedikt Bünz, Dan Boneh, and Zhenfei Zhang. HyperPlonk: Plonk with linear-time prover and high-degree custom gates. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology - EUROCRYPT 2023 - 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Lyon, France, April 23-27, 2023, Proceedings, Part II*, volume 14005 of *Lecture Notes in Computer Science*, pages 499–530. Springer, 2023. 10, 13, 14

[CF08] Giacomo Como and Fabio Fagnani. Average spectra and minimum distances of low-density parity-check codes over abelian groups. *SIAM J. Discret. Math.*, 23(1):19–53, 2008. 23

[CY24] Alessandro Chiesa and Eylon Yogev. *Building Cryptographic Proofs from Hash Functions*. 2024. 6

[DP24] Benjamin E. Diamond and Jim Posen. Proximity testing with logarithmic randomness. *IACR Commun. Cryptol.*, 1(1):2, 2024. 6, 20

[FLY22] Zhiyuan Fan, Jiatu Li, and Tianqi Yang. The exact complexity of pseudorandom functions and the black-box natural proof barrier for bootstrapping results in computational complexity. In Stefano Leonardi and Anupam Gupta, editors, *STOC '22: 54th Annual ACM SIGACT Symposium on Theory of Computing, Rome, Italy, June 20 - 24, 2022*, pages 962–975. ACM, 2022. 5

[Gal62] Robert G. Gallager. Low-density parity-check codes. *IRE Trans. Inf. Theory*, 8(1):21–28, 1962. 8, 21

[Gal63] Robert G. Gallager. *Low-Density Parity-Check Codes*. Monograph. MIT Press, Cambridge, MA, 1963. 8, 21

[GDP73] S. I. Gelfand, R. L. Dobrushin, and M. S. Pinsker. On the complexity of coding. In *Second International Symposium on Information Theory*, pages 177–184, 1973. 5, 7

[GG25] Rohan Goyal and Venkatesan Guruswami. Optimal proximity gaps for subspace-design codes and (random) Reed-Solomon codes. *Electron. Colloquium Comput. Complex.*, TR25-166, 2025. 11

[GLS+23] Alexander Golovnev, Jonathan Lee, Srinath T. V. Setty, Justin Thaler, and Riad S. Wahby. Brakedown: Linear-time and field-agnostic SNARKs for R1CS. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part II*, volume 14082 of *Lecture Notes in Computer Science*, pages 193–226. Springer, 2023. 3, 4, 5, 6, 28

[GNR26] Kobi Gurkan, Andrija Novakovic, and Ron D. Rothblum. Bolt implementation. Available at https://github.com/bcc-research/bolt-rs, 2026. 22, 24, 26

[Hab23] Ulrich Haböck. Brakedown's expander code. *IACR Cryptol. ePrint Arch.*, page 769, 2023. 28

[Irr25] Irreducible. Binius64. Available at https://github.com/binius-zk/binius64, 2025. GitHub commit 21b15d5. 28

[Kil92] Joe Kilian. A note on efficient zero-knowledge proofs and arguments (extended abstract). In S. Rao Kosaraju, Mike Fellows, Avi Wigderson, and John A. Ellis, editors, *Proceedings of the 24th Annual ACM Symposium on Theory of Computing, May 4-6, 1992, Victoria, British Columbia, Canada*, pages 723–732. ACM, 1992. 26

[LFKN92] Carsten Lund, Lance Fortnow, Howard J. Karloff, and Noam Nisan. Algebraic methods for interactive proof systems. *J. ACM*, 39(4):859–868, 1992. 13

[Mic00] Silvio Micali. Computationally sound proofs. *SIAM J. Comput.*, 30(4):1253–1298, 2000. 26

[MRR+20] Jonathan Mosheiff, Nicolas Resch, Noga Ron-Zewi, Shashwat Silas, and Mary Wootters. LDPC codes achieve list decoding capacity. In Sandy Irani, editor, *61st IEEE Annual Symposium on Foundations of Computer Science, FOCS 2020, Durham, NC, USA, November 16-19, 2020*, pages 458–469. IEEE, 2020. 11

[NA25] Andrija Novakovic and Guillermo Angeris. Ligerito: A small and concretely fast polynomial commitment scheme. *IACR Cryptol. ePrint Arch.*, page 1187, 2025. 3, 4, 9, 10, 17, 26, 27

[NST25] Vineet Nair, Ashish Sharma, and Bhargav Thankey. Brakingbase - A linear prover, poly-logarithmic verifier, field agnostic polynomial commitment scheme. In Goichiro Hanaoka and Bo-Yin Yang, editors, *Advances in Cryptology - ASIACRYPT 2025 - 31st International Conference on the Theory and Application of Cryptology and Information Security, Melbourne, VIC, Australia, December 8-12, 2025, Proceedings, Part V*, volume 16249 of *Lecture Notes in Computer Science*, pages 427–455. Springer, 2025. 3

[QGZ26] Wenjie Qu, Yanpei Guo, and Jiaheng Zhang. Lightning, field-agnostic super-efficient polynomial commitment scheme. *IACR Cryptol. ePrint Arch.*, 2026:258, 2026. 6

[Rot24] Ron D. Rothblum. A note on efficient computation of the multilinear extension. Cryptology ePrint Archive, Paper 2024/1103, 2024. https://eprint.iacr.org/2024/1103. 19

[RR24] Noga Ron-Zewi and Ron Rothblum. Local proofs approaching the witness length. *J. ACM*, 71(3):18, 2024. 3, 6, 10, 13

[RR25] Noga Ron-Zewi and Ron Rothblum. Proving as fast as computing: Succinct arguments with constant prover overhead. *J. ACM*, 72(2):15:1–15:54, 2025. 3

[RRR16] Omer Reingold, Ron D. Rothblum, and Guy N. Rothblum. Constant-round interactive proofs for delegating computation. *Electron. Colloquium Comput. Complex.*, TR16-061, 2016. 13

[RVW13] Guy N. Rothblum, Salil P. Vadhan, and Avi Wigderson. Interactive proofs of proximity: delegating computation in sublinear time. In Dan Boneh, Tim Roughgarden, and Joan Feigenbaum, editors, *Symposium on Theory of Computing Conference, STOC'13, Palo Alto, CA, USA, June 1-4, 2013*, pages 793–802. ACM, 2013. 7, 15

[Set20] Srinath T. V. Setty. Spartan: Efficient and general-purpose zksnarks without trusted setup. In Daniele Micciancio and Thomas Ristenpart, editors, *Advances in Cryptology - CRYPTO 2020 - 40th Annual International Cryptology Conference, CRYPTO 2020, Santa Barbara, CA, USA, August 17-21, 2020, Proceedings, Part III*, volume 12172 of *Lecture Notes in Computer Science*, pages 704–737. Springer, 2020. 26, 27

[Spi96] Daniel A. Spielman. Linear-time encodable and decodable error-correcting codes. *IEEE Trans. Inf. Theory*, 42(6):1723–1731, 1996. 5, 7, 11, 25

[Tha13] Justin Thaler. Time-optimal interactive proofs for circuit evaluation. In Ran Canetti and Juan A. Garay, editors, *Advances in Cryptology - CRYPTO 2013 - 33rd Annual Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2013. Proceedings, Part II*, volume 8043 of *Lecture Notes in Computer Science*, pages 71–89. Springer, 2013. 13

[YHC+11] Shengtian Yang, Thomas Honold, Yan Chen, Zhaoyang Zhang, and Peiliang Qiu. Weight distributions of regular low-density parity-check codes over finite fields. *IEEE Trans. Inf. Theory*, 57(11):7507–7521, 2011. 9, 23, 24, 26

[ZCF24] Hadas Zeilberger, Binyi Chen, and Ben Fisch. Basefold: Efficient field-agnostic polynomial commitment schemes from foldable codes. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part X*, volume 14929 of *Lecture Notes in Computer Science*, pages 138–169. Springer, 2024. 3, 27